
CIR Diffusion with Tweedie-Type Score Training

Simplex-aware generative modeling for categorical data

Changkun Guan, Zedong Wang

Roadmap

- Problem Setup
- Literature Review
- Our Method
- Results
- Future Direction

Problem: Discrete Categorical Data

Examples of Categorical Data

- a DNA base: A, C, G, or T
- a Sudoku cell: one digit from 1 to 9

A Common Representation

- A becomes $(1, 0, 0, 0)$
- C becomes $(0, 1, 0, 0)$

Key Point

These vectors are at the vertices of a high dimensional simplex.

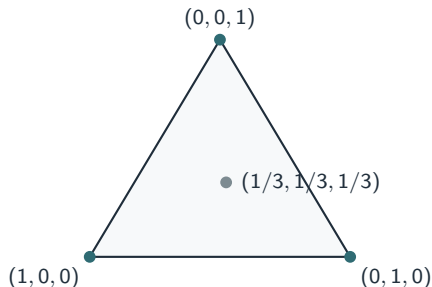
The Simplex, Visually and Intuitively

Definition

$$\Delta^{d-1} = \left\{ x \in \mathbb{R}^d : x_i \geq 0, \sum_{i=1}^d x_i = 1 \right\}.$$

Interpretation

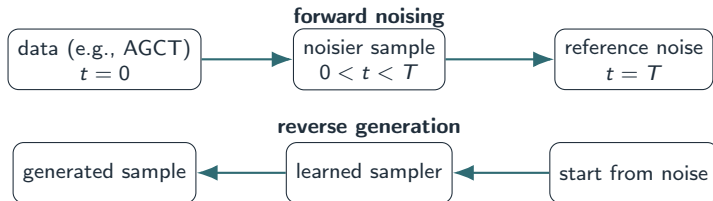
- each vertex is the target we want to sample
- all coordinates are nonnegative
- the total mass is fixed at 1



What Is a Diffusion Model?

Basic picture

- The forward process gradually corrupts data (e.g., some SDE).
- At large time, we reach an easy-to-sample distribution (e.g., Gamma)
- The reverse process learns how to turn noise back into data.



Core challenge: How do we define these forward and reverse steps **efficiently**

Discrete diffusion for categorical data: prior work

Main paradigms

CTMCs | Gaussian relaxations | simplex-based methods | latent methods.

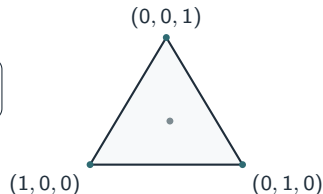
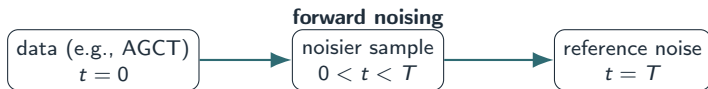
Representative timeline

- **2022: Campbell et al.; Sun et al.** Continuous-time discrete diffusion via CTMC formulations.
- **2022: Richemond et al.** CIR-based simplex diffusion.
- **2023: Avdeyev et al. (DDSM)** Simplex-based diffusion via Jacobi SDE.
- **2023: Dieleman et al.; Li et al.** Latent discrete diffusion in learned continuous representations.

Discrete diffusion for categorical data: prior work

Main paradigms

CTMCs | Gaussian relaxations | **simplex-based methods** | latent methods.



Limitation of prior simplex-based methods



An example Jacobi SDE as the forward process, whose invariant measure is **Beta**(a, b),

$$dX_t = [a(1 - X_t) + bX_t]dt + \sqrt{X_t(1 - X_t)}dW_t$$

$$d\bar{X}_t = \left[\frac{1}{2} (a(1 - \bar{X}_t) - b\bar{X}_t) - (1 - 2\bar{X}_t) - \bar{X}_t(1 - \bar{X}_t) \nabla \log p_t(\bar{X}_t) \right] dt + \sqrt{\bar{X}_t(1 - \bar{X}_t)} d\bar{W}_t.$$

Training the score function requires analytical form as

$$p(x^t | x^0) = \mathcal{B}_{a,b}(x^t) \left(1 + \sum_{n=1}^{\infty} \frac{e^{\lambda_n t}}{d_n} R_n^{(a,b)}(x^0) R_n^{(a,b)}(x^t) \right).$$

- **Tractable only through approximation** implementation requires finite truncation

$$\sum_{n=1}^{\infty} (\dots) \rightsquigarrow \sum_{n=1}^N (\dots),$$

What We Want

We aim at an **efficient, tractable** simplex-based discrete diffusion model.

Efficiency (Empirical)

- Running Time is shorter
- Leverage Deep Learning
- Good Performance on Metrics

Tractability (Theoretical)

- Previous sampling error is not analyzable, e.g., truncation error
- We want tractable errors and provable guarantees

The Diffusion Model We Work with

CIR Process

$$dR_t = (\alpha - \lambda R_t)dt + \sqrt{2R_t}dB_t$$

$$d\overleftarrow{R}_t = \left[(\alpha - 2 - \lambda \overleftarrow{R}_t) - 2\overleftarrow{R}_t \nabla_{\overleftarrow{R}_t} \log p_t(\overleftarrow{R}_t) \right] dt + \sqrt{2\overleftarrow{R}_t} d\overleftarrow{B}_t.$$

Design principle: Avoid the unimplementable transition kernel via interpolation

$$R_t \stackrel{d}{=} \boxed{(1 - e^{-\lambda t}) G} + e^{-\lambda t} \tilde{R}_t, \quad (\text{2017, Arras \& Swan})$$

where $G \sim \text{Gamma}(\alpha, \lambda)$ and

$$\tilde{R}_t \mid R_0 \sim \text{Gamma}(K_t, \lambda_t) \quad K_t \mid R_0 \sim \text{Poisson}(R_0 \lambda_t).$$

Integration By Parts

A conditional expectation representation of score function

$$s_t(R_t) := 1 + R_t \nabla_{R_t} \log p_t(R_t) = \mathbb{E}[(\alpha + K_t - \lambda_t R_t) \mid R_t], \quad a.s.$$

Analogy to Ornstein Uhlenbeck process, $dX_t = -X_t dt + \sqrt{2}dW_t$ where

$$s_t(X_t) := X_t + \nabla_{X_t} \log p_t(X_t) = \mathbb{E}[e^{-t} X_0 - \frac{e^{-2t}}{\sqrt{1 - e^{-2t}}} \mid X_t], \quad a.s.$$

From the geometry of conditional expectation

$$s_t(R_t) = \min_{s \in L^2(R_t)} \mathbb{E}[(\alpha + K(t, Y, \lambda) - \lambda_t R_t - s(R_t))^2]$$

We approximate the L^2 space $\mathcal{S}$ by neural network space

Our Method: From Class Techniques to CIR Design

Class problem	Technique used	Role in our project
Generate samples from a stochastic process	exact random variate / Poisson–Gamma sampling	simulate the CIR forward noising law without Euler discretization
Estimate a hard conditional quantity	conditioning / L^2 projection	turn the score into posterior-count regression
Reduce variance when rare or stiff regions matter	importance sampling / change of measure	oversample small diffusion times where $c_t \sim 1/t$
Train from Monte Carlo samples	empirical risk minimization + SGD	learn the count map with a standard MSE objective

Design principle: preserve simplex geometry, but make score training a supervised regression problem.

Detail 1: CIR Forward Simulation

Problem: forward simulation should not introduce discretization bias.

$$R_t \stackrel{d}{=} (1 - e^{-\lambda t}) G + e^{-\lambda t} \tilde{R}_t,$$

where $G \sim \text{Gamma}(\alpha, \lambda)$ and

$$\tilde{R}_t \mid R_0 \sim \text{Gamma}(K_t, \lambda_t) \quad K_t \mid R_0 \sim \text{Poisson}(R_0 \lambda_t).$$

Design consequence: training pairs are generated by direct simulation approach, not Euler discretization.

Class link: random variate generation; Poisson process and Poisson random-variable simulation.

Detail 2: Small-Time Instability and Importance Sampling

Problem: the small-time region is numerically instable.

$$R_t \stackrel{d}{=} (1 - e^{-\lambda t}) G + e^{-\lambda t} \tilde{R}_t, \quad \tilde{R}_t \mid R_0 \sim \text{Gamma}(K_t, \lambda_t) \quad K_t \mid R_0 \sim \text{Poisson}(R_0 \lambda_t).$$

As $t \downarrow 0$,

$$\lambda_t = \frac{\lambda}{1 - e^{-\lambda t}} \sim \frac{1}{t},$$

Thus both the latent Poisson count and score function learning can become large.

Our solution: Importance Sampling Use a proposal $q(t)$ that puts more mass near small t :

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim p} \mathbb{E} [(K(t, R_0, \lambda) - s_\theta(R_t, t))^2 \mid t] = \mathbb{E}_{t \sim q} \left[\frac{p(t)}{q(t)} \mathbb{E} [(K(t, R_0, \lambda) - s_\theta(R_t, t))^2 \mid t] \right].$$

Design consequence: oversample the dangerous region for better approximation.

Class link: importance sampling / change of measure for variance reduction.

Sudoku Benchmark: High level idea

1. Full Solution

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Used to build supervision

2. Puzzle Clues

5	3	.	.	7	.	.	.	.
6	.	.	1	9	5	.	.	.
.	9	8	.	.	.	.	6	.
8	.	.	.	6	.	.	.	3
4	.	.	8	.	3	.	.	1
7	.	.	.	2	.	.	.	6
.	6	.	.	.	.	2	8	.
.	.	.	4	1	9	.	.	5
.	.	.	.	8	.	.	7	9

Hide entries to make clues

3. Noisy State x_t

Diffuse the same full solution.

Highlighted entry:

4

After forward diffusion:

$4 \rightarrow (0.07, 0.05, 0.06, 0.55, 0.05, 0.04, 0.07, 0.05, 0.06)$

The largest entry still tends to stay near the true digit.

Model input: (puzzle clues, x_t)

Supervision target: posterior counts from the full solution

Sudoku Benchmark: Forward Process

1. Start from the clean board

y = full solved board, $c = (m, v)$ = puzzle clues.

y is the clean board used by the forward CIR diffusion.

2. Noisy board at time t

$p_t(\cdot | y)$ = distribution of the noised board at time t

when the process starts at y at time 0.

So at any chosen time t ,

$$x_t \sim p_t^{\text{CIR}}(\cdot | y).$$

3. How we use it in training

We do **not** need the whole sample path. Instead:

- 1 choose one time t ,
- 2 draw one noisy snapshot x_t from the exact transition law,
- 3 train on that one snapshot.

Sudoku Benchmark: Exact Draw

1. Exact draw used in code

For each cell-digit coordinate, the exact CIR transition uses

- one latent Poisson draw,
- then one Gamma draw conditional on that count.

This gives one exact sample x_t from the one-time transition law.

Takeaway

The Poisson-Gamma step is how we draw one exact noisy training snapshot x_t from the forward process.

Sudoku Benchmark: Model Input

1. Model input

After drawing x_t , the network sees

$$(x_t, t, c),$$

where

$$c = (m, v),$$

with m indicating which cells are observed clues and v giving the fixed digits in those cells.

Takeaway

The network input combines the noisy board x_t , the diffusion time t , and the puzzle-clue information c .

Sudoku Benchmark: Model Target

1. What the network predicts

The network does **not** output the solved board directly. It predicts

$$\hat{g}_{\theta}(x_t, t, c) \approx \log(1 + \eta_t(x_t, c)),$$

where

$$\eta_t(x_t, c) = \mathbb{E}[K \mid x_t, c].$$

2. What is K ?

K is the latent Poisson count from the exact forward CIR transition. So the supervision target is the posterior expected count

$$\eta_t(x_t, c) = \mathbb{E}[K \mid x_t, c].$$

3. Intuition

This output is one continuous 9-way vector for each Sudoku cell.

For one cell, a large value on digit 4 means:

given this noisy cell and these clues, digit 4 looks most consistent with the hidden clean board.

4. Why this target

We train on this posterior-count target because reverse diffusion needs a score field. In CIR, we recover that score analytically from the predicted expected counts.

So the network output is **denoising guidance**, not the Sudoku board itself.

Sudoku Benchmark: Reverse Sampling

1. Recover the score

From the network output we recover

$$\hat{\eta}_{\theta}(x_t, c) = \exp(\hat{g}_{\theta}) - 1,$$

and then recover the reverse-time score analytically from $\hat{\eta}_{\theta}$.

2. Run reverse diffusion

- 1 Start from reference noise $R_T \sim \pi$.
- 2 At each reverse step, evaluate the learned score using the clues c .
- 3 Update the state and re-impose the clue cells.

3. Output of the sampler

After many reverse steps, we do **not** yet have digits. We have

cellwise probabilities for all 81 cells.

Each cell has a 9-way probability vector over digits $1, \dots, 9$.

So reverse diffusion turns the learned guidance field into a probabilistic Sudoku board, not yet an integer solution.

Sudoku Benchmark: Decoding to a Board

1. Start from the sampler output

Input to the decoder:

81 cellwise 9-way probability vectors.

So for each cell we know a probability distribution over digits $1, \dots, 9$.

2. Convert probabilities to digits

- 1 choose digits from the final cellwise probabilities,
- 2 keep all clue cells fixed,
- 3 fill the missing cells only.

3. Local repair step

After the first pass, the board may still have column or box conflicts.

So the benchmark applies a matched local repair decoder to reduce those conflicts while respecting the fixed clues.

4. Final output

The final result is one completed integer Sudoku board, which we then evaluate by:

- **missing-cell accuracy:** among the cells blanked out by the puzzle, the fraction filled with the correct digit,
- **full-board validity:** the fraction of completed boards that satisfy every row, column, and 3×3 box constraint,

Sudoku Benchmark: Result Table

Method	Missing Acc. $\uparrow$	Board Validity $\uparrow$	Small- t MSE $\downarrow$	Time (min) $\downarrow$
CIR Tweedie	0.747 $\pm$ 0.003	0.412 $\pm$ 0.005	0.067 $\pm$ 0.012	58.6 $\pm$ 1.6
Dirichlet Score	0.741 $\pm$ 0.019	0.371 $\pm$ 0.024	1.317 $\pm$ 0.116	65.6 $\pm$ 5.8

How to read the table

- Missing accuracy is the fraction of hidden cells recovered correctly.
- Small- t MSE is score error on the earliest 20% of sampled times, so lower is better near the clean-data end of reverse sampling.
- In these saved runs, board validity and exact solve coincide because test puzzles are unique-solution and clue cells stay fixed.

Computational setup

- mean $\pm$ std over seeds 0, 1, 2
- clue counts 30 and 40; 2 decoder restarts
- PACE gpu-v100
- 1 V100 GPU, 8 CPU cores, 32 GB RAM per run

Sudoku Benchmark: Summary

Computational takeaway

The Sudoku benchmark supports the claim that

- with the same clue budget and decoder family, CIR completes more cells correctly
- CIR also reduces early-time score error from 1.317 to 0.067
- CIR produces more valid full boards; in these saved runs, that is also an exact-solve improvement
- CIR is also faster overall in this benchmark

DDSM DNA Benchmark

One benchmark example

$(0.1, 0.3, 1.2, 2.4, \dots)$
biological signal track

paired with

$A \ C \ G \ T \ \dots$
real DNA sequence from the dataset

What the task asks

Given the biological signal track, generate a **full DNA sequence** whose predicted biology is close to the real DNA sequence in the benchmark.

signal track $\Rightarrow$ generate DNA $\Rightarrow$ evaluate biology

DDSM DNA Benchmark: How Training Works

- 1 Start from a real DNA sequence in the benchmark.
- 2 Add diffusion noise to get a noisy sequence x_t .
- 3 Give the model the triple $(x_t, t, \text{signal track})$.
- 4 Train it on the method-specific score target.

CIR Tweedie/count model

Use CIR noising, train on posterior expected counts, and recover the score analytically.

Direct Dirichlet score baseline

Use product-Dirichlet noising and train the network to predict Dirichlet concentrations, which determine the baseline score.

What is matched?

Same DNA split, conditioning signal, backbone family, seeds, and evaluation pipeline. What changes is the forward noising rule and the score-learning target.

DDSM DNA Benchmark: How Testing Works

- 1 Keep the signal track for a held-out test DNA example.
- 2 Start from random reference noise.
- 3 Run reverse diffusion to generate a full DNA sequence.
- 4 Compare the generated sequence to the real test example through a pretrained biology model.

Main evaluation metric

Run both the real and generated DNA through pretrained SEI, extract the target feature activity, and compare them with **SP-MSE**.

DDSM DNA Benchmark: Result Table

Method	SP-MSE ↓	Total Time (min) ↓	Score MSE ↓	Instability ↓
CIR Tweedie	0.117 ± 0.006	99.6 ± 0.2	0.061 ± 0.000	1.000 ± 0.000
Dirichlet Score	0.116 ± 0.001	71.7 ± 19.4	0.416 ± 0.225	0.365 ± 0.044

What CIR clearly improves

- much smaller small-time error
- much smaller score MSE
- cleaner score-learning target

What CIR does not yet improve

- not better headline SP-MSE
- not faster end to end
- not more stable reverse sampling

Computational takeaway

The DNA benchmark is considered as a large benchmark than the previous Sudoku.

- the CIR Tweedie/count target gives a much cleaner score-learning signal
- it remains competitive on the headline biological metric
- but more experiments with more seed and more evaluation metrics (current reported one is H3K4me3) are still needed for a clear comparison.